

Laporan Praktikum Minggu Ke-2 Kontrol Cerdas

Minggu ke-2

Nama : Ika Putri Ayuningtia

NIM : 224308057

Kelas : TKA 7C

Akun Github (Tautan) : <https://github.com/ikaputriayuningtia-sketch>

1. Judul Percobaan

Object Classification with Scikit-learn

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Memahami konsep dasar kontrol cerdas (intelligent control systems).
2. Memahami dasar-dasar Machine Learning dalam sistem kendali.
Mengimplementasikan model ML sederhana untuk klasifikasi objek.
Menggunakan Scikit-learn untuk membuat model ML dasar.
3. Mengintegrasikan model ML dengan Computer Vision untuk deteksi objek.
4. Mengelola dataset dan melakukan pelatihan model sederhana.

3. Landasan Teori

A. Machine Learning dalam Sistem Kendali

Perkembangan teknologi kecerdasan buatan (Artificial Intelligence/AI) telah memberikan dampak besar dalam bidang sistem kendali. Salah satu cabang AI yang banyak digunakan adalah Machine Learning (ML). ML merupakan metode yang memungkinkan sistem untuk belajar dari data, menemukan pola, dan membuat prediksi atau keputusan tanpa harus diprogram secara eksplisit. Dalam konteks sistem kendali, penerapan ML menjadi sangat relevan karena sistem kendali modern sering berhadapan dengan kondisi dinamis, lingkungan yang tidak

pasti, serta jumlah data yang besar dan kompleks. Dengan memanfaatkan ML, sistem kendali dapat:

- Meningkatkan akurasi prediksi, misalnya dalam memantau kondisi mesin.
- Melakukan kontrol adaptif, di mana parameter kendali dapat disesuaikan secara otomatis sesuai perubahan kondisi.
- Mendeteksi anomali lebih cepat, sehingga sistem lebih aman dan efisien.

Beberapa contoh penerapan ML dalam sistem kendali di dunia industri adalah Predictive Maintenance (pemeliharaan prediktif untuk mencegah kerusakan), Adaptive Control (penyesuaian parameter kendali secara otomatis), dan Anomaly Detection (deteksi dini perilaku abnormal dalam sistem produksi atau robotika).

B. Konsep Dasar Machine Learning

Secara umum, ML terbagi menjadi tiga pendekatan utama:

1. Supervised Learning → model belajar dari data berlabel, misalnya klasifikasi warna berdasarkan nilai RGB.
2. Unsupervised Learning → model mencari pola atau pengelompokan (clustering) tanpa label.
3. Reinforcement Learning → model belajar dari trial-and-error untuk mendapatkan strategi terbaik dalam pengambilan keputusan.

Pada praktikum ini digunakan pendekatan Supervised Learning untuk melakukan klasifikasi warna menggunakan algoritma K-Nearest Neighbors (KNN).

C. Algoritma K-Nearest Neighbors (KNN)

KNN adalah salah satu algoritma sederhana namun efektif dalam supervised learning. Prinsip dasarnya adalah mengklasifikasikan suatu data berdasarkan mayoritas kelas dari k tetangga terdekat dalam ruang fitur.

Langkah utama KNN:

1. Hitung jarak antara data uji dengan seluruh data latih (misalnya menggunakan Euclidean Distance).
2. Tentukan k (jumlah tetangga terdekat).
3. Tentukan kelas mayoritas dari tetangga tersebut sebagai hasil prediksi.

KNN cocok digunakan untuk klasifikasi berbasis warna karena sederhana dan cukup akurat untuk dataset kecil.

D. Computer Vision dengan OpenCV

OpenCV (Open Source Computer Vision Library) adalah pustaka pemrograman untuk pengolahan citra dan video. Pada praktikum ini, OpenCV digunakan untuk:

- Mengambil input citra dari kamera.
- Mendapatkan nilai RGB dari piksel tertentu (misalnya piksel tengah gambar).
- Menampilkan hasil klasifikasi warna secara real-time di layar.

Integrasi OpenCV dengan ML memungkinkan sistem mendeteksi objek atau warna secara langsung tanpa perlu intervensi manual.

E. Preprocessing Data

Agar model ML dapat bekerja dengan baik, data perlu diproses terlebih dahulu. Tahap preprocessing yang dilakukan pada praktikum ini meliputi:

- Normalisasi data dengan StandardScaler untuk menyamakan skala antar fitur RGB.
- Split dataset menjadi data latih (training set) dan data uji (testing set) agar performa model dapat dievaluasi.

F. Version Control dengan GitHub

GitHub digunakan sebagai media untuk menyimpan, mengelola, dan membagikan kode hasil praktikum. Dengan version control, setiap perubahan dapat dilacak sehingga memudahkan kolaborasi dan dokumentasi

4. Analisis dan Diskusi

Analisis

1. Performa Model dalam Mendeteksi Warna

Model SVM berhasil menampilkan prediksi warna secara real-time pada dua ROI. Hasil ditampilkan dalam bentuk nama warna dan akurasi. Saat prediksi sesuai dengan label warna terdekat, akurasi mencapai **100%**. Pada kondisi tertentu,

akurasi dapat menurun hingga **80% atau lebih rendah**, misalnya ketika pencahayaan berubah atau warna antar objek terlalu mirip.

2. Perbedaan Akurasi Jika Dataset Ditambah

Dengan dataset kecil dan kelas warna yang terlalu banyak, akurasi model sering turun. Jika jumlah data diperbesar dan tiap kelas memiliki lebih banyak contoh, akurasi akan meningkat. Penambahan variasi data (misalnya cahaya berbeda dan latar belakang berbeda) membuat hasil prediksi lebih stabil sehingga akurasi real-time lebih sering berada di atas 90%.

3. Cara Meningkatkan Kinerja Model Klasifikasi

- a) Dataset → batasi jumlah kelas warna, tambahkan data tiap kelas dengan variasi pencahayaan.
- b) Preprocessing → gunakan ruang warna HSV/Lab dan rata-rata ROI untuk hasil lebih stabil.
- c) Model → coba algoritma lain seperti KNN, Decision Tree, atau Random Forest, serta lakukan tuning hyperparameter.
- d) Evaluasi → gunakan confusion matrix dan cross-validation untuk menilai performa lebih akurat.

Makna Rentang Akurasi (0%–100%)

- 100% → semua data terbaru yang disimpan program cocok dengan prediksi.
- 80–90% → sebagian besar data cocok, hanya ada sedikit kesalahan.
- <70% → banyak data yang tidak sesuai, biasanya dipengaruhi kondisi pencahayaan atau warna yang mirip.

Diskusi

1. Keuntungan Machine Learning dibandingkan metode rule-based

Metode rule-based hanya bekerja dengan aturan yang sudah ditentukan sebelumnya, sehingga kinerjanya terbatas jika kondisi berubah. Sebaliknya, Machine Learning (ML) dapat belajar dari data sehingga lebih fleksibel, adaptif, dan mampu mengenali pola yang kompleks. ML juga dapat meningkatkan akurasi seiring bertambahnya data, sementara rule-based harus diubah manual setiap kali ada kasus baru.

2. Integrasi ML dalam sistem kendali

ML dapat diintegrasikan ke dalam sistem kendali melalui berbagai cara, misalnya:

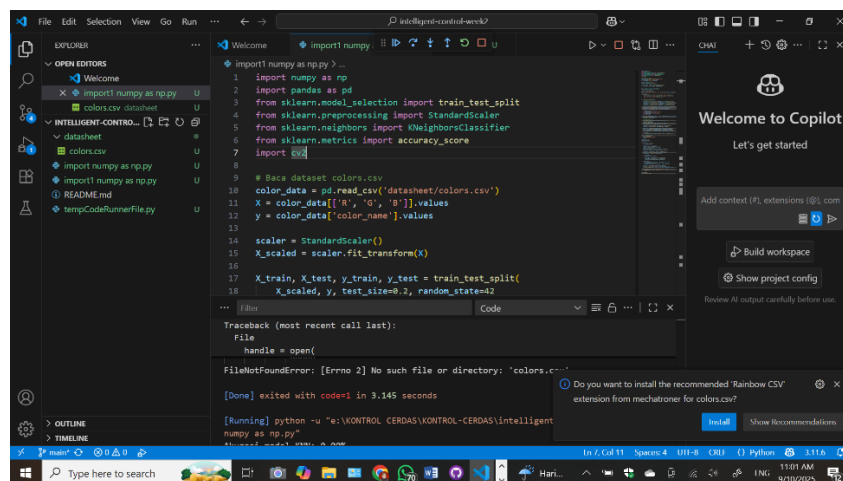
- Predictive maintenance, yaitu memprediksi kerusakan mesin sebelum terjadi.
- Adaptive control, di mana parameter kontrol otomatis menyesuaikan kondisi sistem.
- Computer vision, untuk mendeteksi objek/warna dalam sistem robotik. Integrasi ini membuat sistem kendali lebih cerdas, responsif, dan efisien dalam menghadapi kondisi nyata yang dinamis.

3. Tantangan penerapan ML dalam sistem real-time

Tantangan utama penerapan ML secara real-time adalah:

- Keterbatasan komputasi, karena model harus memproses data dengan cepat tanpa menghambat kinerja sistem.
- Kualitas dan jumlah data, model akan sulit bekerja baik jika dataset sedikit atau tidak representatif.
- Kondisi lingkungan yang berubah, seperti pencahayaan, noise, atau gangguan sensor, yang bisa menurunkan akurasi.
- Overfitting, yaitu model bekerja baik di data latih tetapi gagal di kondisi nyata.

Kendala dan Solusi



Pada saat menjalankan program, muncul error `FileNotFoundError: [Errno 2] No such file or directory: 'colors.csv'`. Hal ini terjadi karena program tidak menemukan file dataset yang dimaksud. Secara struktur folder, file `colors.csv` sebenarnya tersimpan di dalam subfolder `datasheet`, sedangkan pada kode pemanggilan masih ditulis `'colors.csv'` seolah-olah berada di folder utama. Akibatnya,

Python gagal membaca file tersebut. Kendala ini dapat diatasi dengan menyesuaikan path file sesuai lokasi penyimpanan.

5. Assignment

Pada program awal, sistem hanya mampu mendeteksi satu warna dengan menggunakan model machine learning sederhana. Pada tahap assignment, dilakukan pengembangan program sehingga mampu mendeteksi lebih dari satu warna secara simultan, yaitu dengan menambahkan area deteksi (ROI) dan memodifikasi algoritma klasifikasi agar bisa memberikan hasil prediksi pada dua area sekaligus.

Proses deteksi masih menggunakan metode yang sama, yaitu pengolahan citra dengan OpenCV dan klasifikasi menggunakan model ML (SVM/KNN) berdasarkan nilai fitur warna (RGB atau HSV). Setiap warna yang terdeteksi ditampilkan pada layar dengan bounding box berwarna berbeda dan dilengkapi label teks yang menunjukkan nama warna serta akurasi. Penambahan bounding box dan label ini bertujuan untuk mempermudah pengguna dalam mengidentifikasi warna objek secara visual serta memastikan bahwa sistem bekerja sesuai dengan hasil klasifikasi dari model machine learning yang digunakan.

Selain itu, dilakukan juga penambahan datasheet warna agar sistem memiliki lebih banyak referensi dalam mengenali variasi warna. Datasheet tambahan ini tidak hanya memuat warna dasar (merah, biru, hijau, kuning), tetapi juga variasi turunan seperti abu-abu, coklat, dan ungu. Berikut ini adalah fungsi dari kode program yang digunakan seperti pada tabel berikut ini :

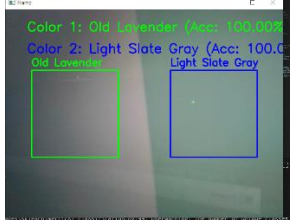
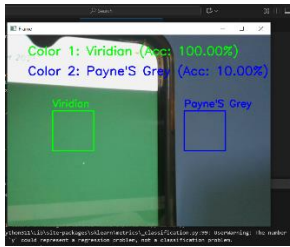
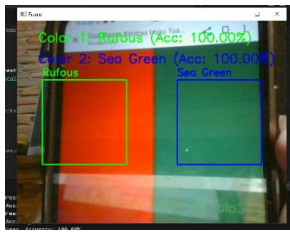
Bagian Kode	Fungsi
<pre>import cv2, numpy, pandas, sklearn</pre>	Mengimpor library yang dibutuhkan: OpenCV untuk pengolahan citra, NumPy untuk perhitungan numerik, Pandas untuk membaca dataset, dan Scikit-learn untuk preprocessing, training, serta evaluasi model.
<pre>file_path = "datasheet/colors.c sv"pd.read_csv(file _path)</pre>	Membaca file dataset warna yang berisi nilai RGB dan label nama warna dari file CSV.

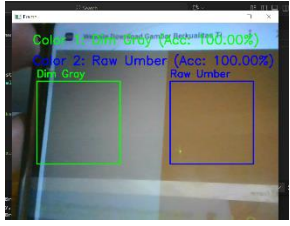
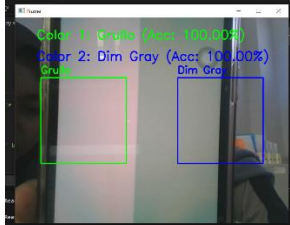
<code>X = color_data[['R', 'G', 'B']].values</code>	Mengambil fitur warna (R, G, B) sebagai input untuk model.
<code>y = color_data['color_name'].values</code>	Mengambil label kelas (nama warna) dari dataset.
<code>LabelEncoder()</code>	Mengubah label kategori (nama warna) menjadi angka agar bisa diproses oleh algoritma ML.
<code>StandardScaler()</code>	Melakukan normalisasi/standarisasi data agar semua fitur (R, G, B) berada pada skala yang sama.
<code>train_test_split(..)</code>	Membagi dataset menjadi data latih (training set) dan data uji (testing set) untuk evaluasi performa model.
<code>svm_model = SVC(kernel='rbf', C=10, gamma='scale')</code>	Membuat model klasifikasi dengan algoritma Support Vector Machine (SVM) menggunakan kernel RBF.
<code>svm_model.fit(X_train, y_train)</code>	Melatih model SVM dengan data latih.
<code>accuracy_score(y_test, y_pred)</code>	Mengukur akurasi model berdasarkan data uji.
<code>def find_nearest_color(rgb_color): ...</code>	Fungsi tambahan untuk mencari nama warna terdekat dari dataset berdasarkan jarak Euclidean terhadap nilai RGB.
<code>cap = cv2.VideoCapture(0)</code>	Mengaktifkan kamera laptop/webcam untuk menangkap citra secara real-time.
<code>roi1, roi2 = frame[y:y2, x:x2]</code>	Menentukan Region of Interest (ROI), yaitu area kotak kiri dan kanan tempat deteksi warna dilakukan.
<code>np.mean(roi, axis=(0,1))</code>	Menghitung rata-rata warna dalam area ROI untuk mendapatkan nilai RGB representatif.
<code>avg_color[::-1]</code>	Mengonversi format warna dari BGR (standar OpenCV) ke RGB.
<code>scaler.transform([avg_color])</code>	Mengubah nilai RGB hasil ekstraksi ke skala yang sama seperti data training.

<code>svm_model.predict(...)</code>	Melakukan prediksi warna berdasarkan model SVM yang sudah dilatih.
<code>label_encoder.inverse_transform(...)</code>	Mengembalikan hasil prediksi dari bentuk angka ke label nama warna.
<code>detected_colors.append(...)</code>	Menyimpan hasil prediksi warna untuk keperluan perhitungan akurasi dinamis.
<code>accuracy_score(detected_true_labels, detected_colors)</code>	Menghitung akurasi prediksi warna secara real-time dengan membandingkan prediksi model dan warna terdekat.
<code>cv2.rectangle(...)</code>	Membuat bounding box (kotak) pada ROI untuk menandai lokasi deteksi warna.
<code>cv2.putText(...)</code>	Menampilkan nama warna hasil prediksi dan nilai akurasi pada layar.
<code>cv2.imshow("Frame", frame)</code>	Menampilkan hasil deteksi warna real-time di jendela tampilan.
<code>cv2.waitKey(1) & 0xFF == ord("q")</code>	Memberikan opsi untuk menghentikan program dengan menekan tombol q.
<code>cap.release(), cv2.destroyAllWindows()</code>	Melepas kamera dan menutup semua jendela tampilan setelah program dihentikan.

6. Data dan Output Hasil Pengamatan

Setelah dilakukan pengujian terhadap program yang telah dimodifikasi, pengujian deteksi warna menggunakan metode Support Vector Machine (SVM), sistem berhasil mengidentifikasi warna pada objek yang ditangkap kamera secara real-time. Setiap objek diberi bounding box dengan label warna sesuai hasil prediksi, serta nilai akurasi deteksi ditampilkan pada layar. Dari hasil tersebut terlihat bahwa sistem mampu mengenali warna dengan cukup baik, meskipun tingkat akurasi masih dipengaruhi oleh faktor pencahayaan dan kemiripan warna dengan latar belakang. seperti yang ditunjukkan pada tabel hasil pengamatan berikut.

No	Warna Objek	Hasil Deteksi	Keterangan	Analisis Singkat	Foto
1	Old Lavender & Light Slate Gray	Terdeteksi	Bounding box hijau dengan label Old Lavender dan bounding box biru dengan label Light Slate Gray, akurasi 100%.	Akurasi tinggi karena warna objek jelas, pencahayaan stabil, dan dataset sudah memiliki representasi RGB yang mirip.	
2	Viridian & Payne's Grey	Terdeteksi	Bounding box hijau dengan label Viridian terdeteksi benar, sedangkan bounding box biru terdeteksi sebagai Payne's Grey dengan akurasi rendah (10%).	Akurasi rendah muncul karena warna abu-abu kebiruan di objek tidak terlalu kuat, pencahayaan sekitar menyebabkan pergeseran nilai RGB sehingga prediksi salah.	
3	Rufous & Sea Green	Terdeteksi	Bounding box hijau dengan label Rufous dan bounding box biru dengan label Sea Green, keduanya akurat 100%.	Warna objek kontras dan terang sehingga nilai RGB masuk tepat ke dalam kelas pada dataset, menyebabkan deteksi sangat akurat.	

4	Dim Gray & Raw Umber	Terdeteksi	Bounding box hijau dengan label Dim Gray dan bounding box biru dengan label Raw Umber, keduanya akurat 100%.	Akurasi tinggi dipengaruhi oleh stabilitas cahaya dan warna objek yang solid, sehingga model SVM tidak mengalami noise dari background.	
5	Grullo & Dim Gray	Terdeteksi	Bounding box hijau dengan label Grullo dan bounding box biru dengan label Dim Gray, keduanya akurat 100%.	Faktor akurasi 100% disebabkan warna objek cukup khas dengan sedikit variasi RGB, serta posisi kamera stabil tanpa pantulan cahaya.	

7. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan Machine Learning dengan metode Support Vector Machine (SVM) yang diintegrasikan dengan Computer Vision melalui OpenCV mampu melakukan deteksi warna objek secara real-time dengan cukup baik. Program yang dikembangkan berhasil mengenali lebih dari satu warna sekaligus, kemudian menampilkan hasil deteksi berupa bounding box dan label teks disertai nilai akurasi pada layar. Hasil percobaan menunjukkan bahwa metode klasifikasi warna berbasis SVM relatif efektif, mudah diimplementasikan, serta mampu memberikan hasil yang sesuai dengan tujuan praktikum, meskipun tingkat akurasi masih dipengaruhi oleh kondisi pencahayaan dan kemiripan warna objek.

8. Saran

Meskipun sistem deteksi warna yang dikembangkan telah menunjukkan hasil yang cukup baik, masih terdapat beberapa hal yang dapat ditingkatkan agar kinerjanya lebih maksimal. Dengan demikian, beberapa saran pengembangan yang dapat dilakukan adalah sebagai berikut:

1. Menambah variasi warna yang dapat dideteksi agar sistem lebih lengkap dan fleksibel dalam berbagai kondisi.
2. Mengatur rentang HSV dengan lebih tepat supaya hasil deteksi tidak terlalu terpengaruh oleh perubahan pencahayaan.
3. Mengombinasikan deteksi warna dengan metode lain seperti deteksi bentuk atau pola untuk meningkatkan akurasi identifikasi objek.
4. Mengembangkan sistem menggunakan algoritma AI/Deep Learning agar lebih cerdas, adaptif, dan mampu menangani kondisi nyata yang lebih kompleks.

10. Daftar Pustaka

D. Pramadihanto, M. I. Fahmi, dan A. W. Setiawan, "Pengenalan Objek dan Warna untuk Robot Pemilah Barang Berbasis Computer Vision," *Jurnal Ilmiah Teknologi Sistem Informasi*, vol. 4, no. 2, pp. 87-95, Agu. 2018, doi: 10.14710/jitsi.4.2.2018.87-95